INTECH
PT Industri Teknologi Informasi dan Komunikasi

Pelatihan Software Engineering

Materi 8
Menyusun Spesifikasi Kebutuhan PL (Sequence Diagram – Class Diagram)



Immah Inayati, S.Kom., M.Kom., MBA

SKKNI YANG AKAN DIBAHAS

Kode Unit : J.62SAD00.007.1

Judul Unit : Membuat Spesifikasi Kebutuhan Software Environment

Deskripsi Unit : Unit kompetensi ini berhubungan dengan pengetahuan, keterampilan, dan sikap kerja yang dibutuhkan dalam membuat spesifikasi kebutuhan software environment.

Elemen Unit :

1. Menyusun spesifikasi kebutuhan software environment
2. Menyusun dokumen sistem

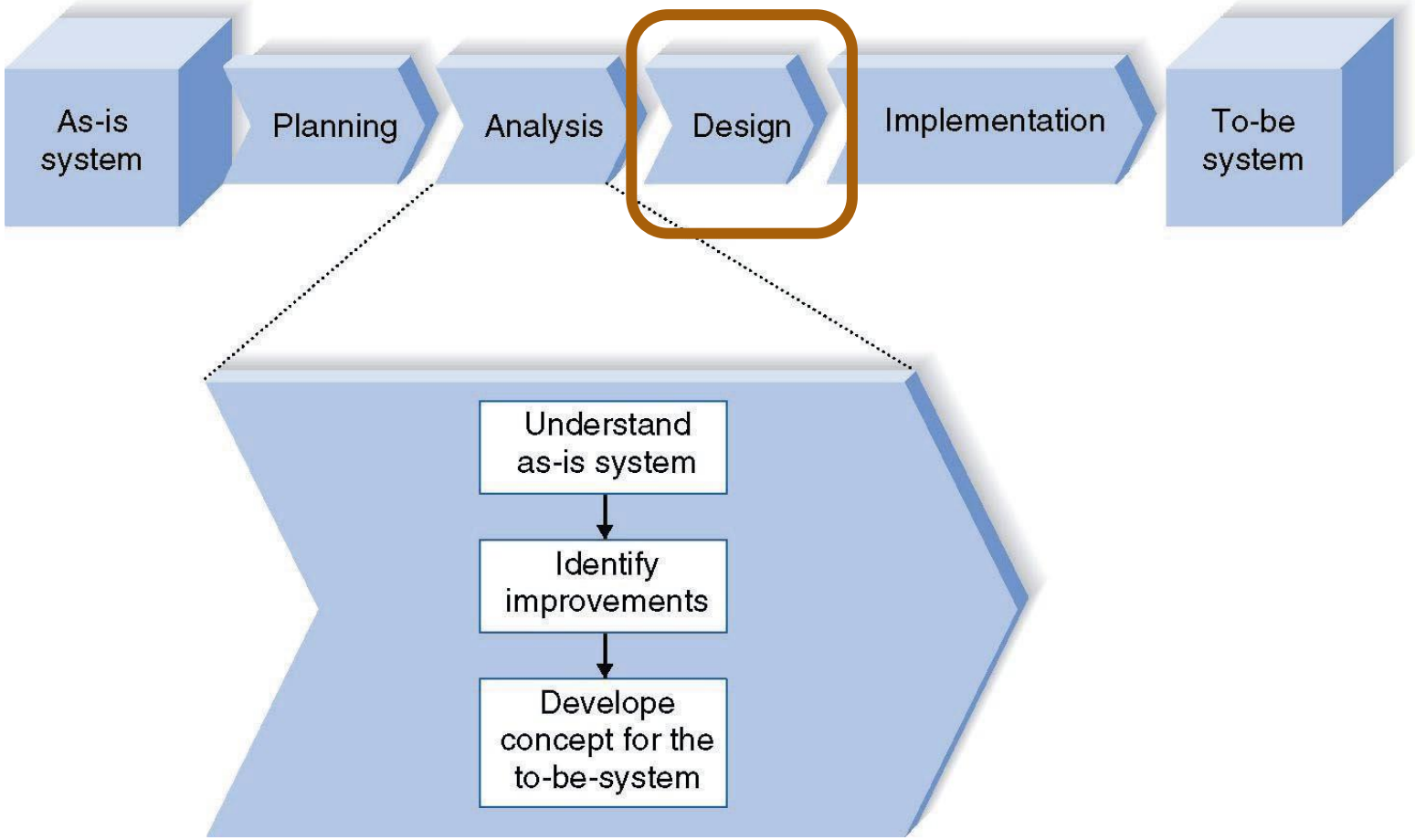
Target Pembahasan

1. Review SDLC
2. Konsep Object – Oriented (Model-View-Controller)
3. Sequence Diagram
4. Class Diagram
5. Praktek

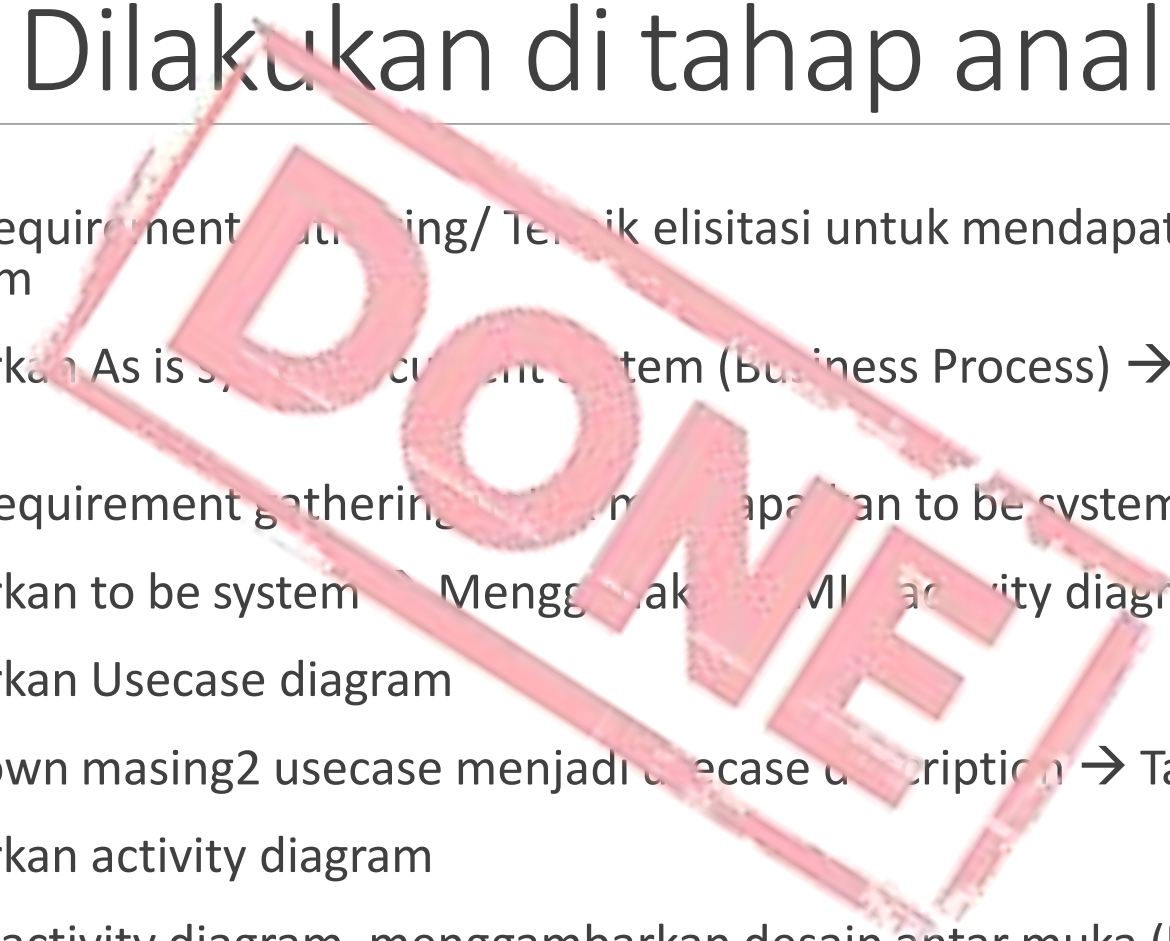
Membangun Software = Membangun rumah



Business Process Analysis Steps



Yang Dilakukan di tahap analisa



1. Melakukan Requirement gathering/ Teknik elisitasi untuk mendapatkan gambaran proses bisnis dari current system
2. Menggambarkan As is system (Business Process) → Menggunakan UML : activity diagram
3. Melakukan Requirement gathering untuk mendapatkan to be system
4. Menggambarkan to be system → Menggunakan UML activity diagram
5. Menggambarkan Usecase diagram
6. Membreakdown masing2 usecase menjadi usecase description → Tahap ini bisa dilewati
7. Menggambarkan activity diagram
8. Untuk setiap activity diagram, menggambarkan desain antar muka (User Interface)

Yang Dilakukan di tahap Desain

1. Dari Activity diagram dan desain antar muka yang dibuat di tahap analisa, dilakukan breakdown menjadi Sequence diagram
2. Menggambarkan Class Diagram
3. Melakukan Permodelan data
4. Mendesain User Interface (User Experience)
5. Mendesain Arsitektur sistem

Target Pembahasan

1. Review SDLC
2. Konsep Object – Oriented (Model-View-Controller)
3. Sequence Diagram
4. Class Diagram
5. Praktek

Object Oriented

Oriented Programming adalah paradigma pemrograman atau klasifikasi yang mengatur sekelompok atribut data dengan fungsi atau metode ke dalam suatu unit, yang dikenal sebagai objek

Objek : seperti dunia nyata: memiliki sifat (data) dan bisa melakukan sesuatu (Method).

Setiap Objek / Class memiliki :

- Data / atribut → menggambarkan sifat
- Fungsi / method → menggambarkan perilaku atau tindakan

Manusia
- Kepala - Tangan - Kaki
- Berjalan - Bernafas - Berbicara

4 konsep dasar OOP

- Enkapsulasi: Mengikat data (atribut) dan metode (fungsi) yang beroperasi pada data tersebut ke dalam satu unit (kelas), sekaligus menyembunyikan detail internal dari dunia luar. Ini membantu melindungi integritas data.
- Pewarisan (Inheritance): Memungkinkan kelas baru (kelas anak) untuk mewarisi atribut dan metode dari kelas yang sudah ada (kelas induk), sehingga mengurangi duplikasi kode dan mempromosikan penggunaan kembali kode.
- Polimorfisme: Berarti "banyak bentuk". Ini adalah kemampuan objek dari kelas yang berbeda untuk merespons pesan yang sama dengan cara yang berbeda. Contohnya, metode makan pada objek "Kucing" dan "Anjing" akan memiliki implementasi yang berbeda.
- Abstraksi: Menyembunyikan detail implementasi yang kompleks dan hanya menampilkan fungsionalitas yang diperlukan kepada pengguna. Ini menyederhanakan cara berinteraksi dengan objek, seperti menyalakan TV tanpa perlu tahu cara kerjanya.

Bahasa Pemrograman Berorientasi Object

1. Java
2. C++
3. C# (C-Sharp)
4. Python
5. PHP – Konsep MVC
6. Ruby
7. Kotlin
8. Javascript

Framework yang menggunakan MVC

- Python:

  Django, Turbogears2, Watson-Framework, dst.

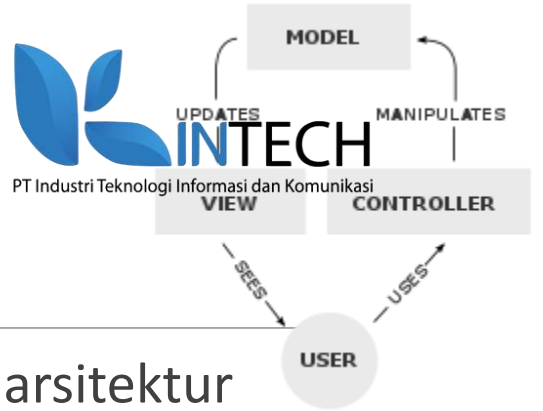
- PHP:

  Laravel, Symfony, CakePHP, Zend, Codeigniter Framework

- Nodejs:

  Express, Adonis, Sails.js, Total.js, Mean.js, Mojito.

Model – View Controller



Model View Controller atau yang dapat disingkat MVC adalah sebuah pola arsitektur dalam membuat sebuah aplikasi dengan cara memisahkan kode menjadi tiga bagian yang terdiri dari:

- **Model**
  Bagian yang bertugas untuk menyiapkan, mengatur, memanipulasi, dan mengorganisasikan data yang ada di database.
- **View**
  Bagian yang bertugas untuk menampilkan informasi dalam bentuk tam[ilan antar muka / *Graphical User Interface* (GUI).
- **Controller**
  Bagian yang bertugas untuk menghubungkan serta mengatur model dan view agar dapat saling terhubung.

MODEL

- Tugas utama: Mengelola data, aturan bisnis, dan logika aplikasi.
- Berinteraksi dengan: Basis data (database) atau sumber data lainnya.
- Contoh:Dalam sistem informasi perpustakaan, Model mengatur data buku, data anggota, dan transaksi peminjaman.
- Fungsi utama Model:
  - Menyimpan dan memvalidasi data.
  - Mengambil data dari database.
  - Mengupdate data saat terjadi perubahan.

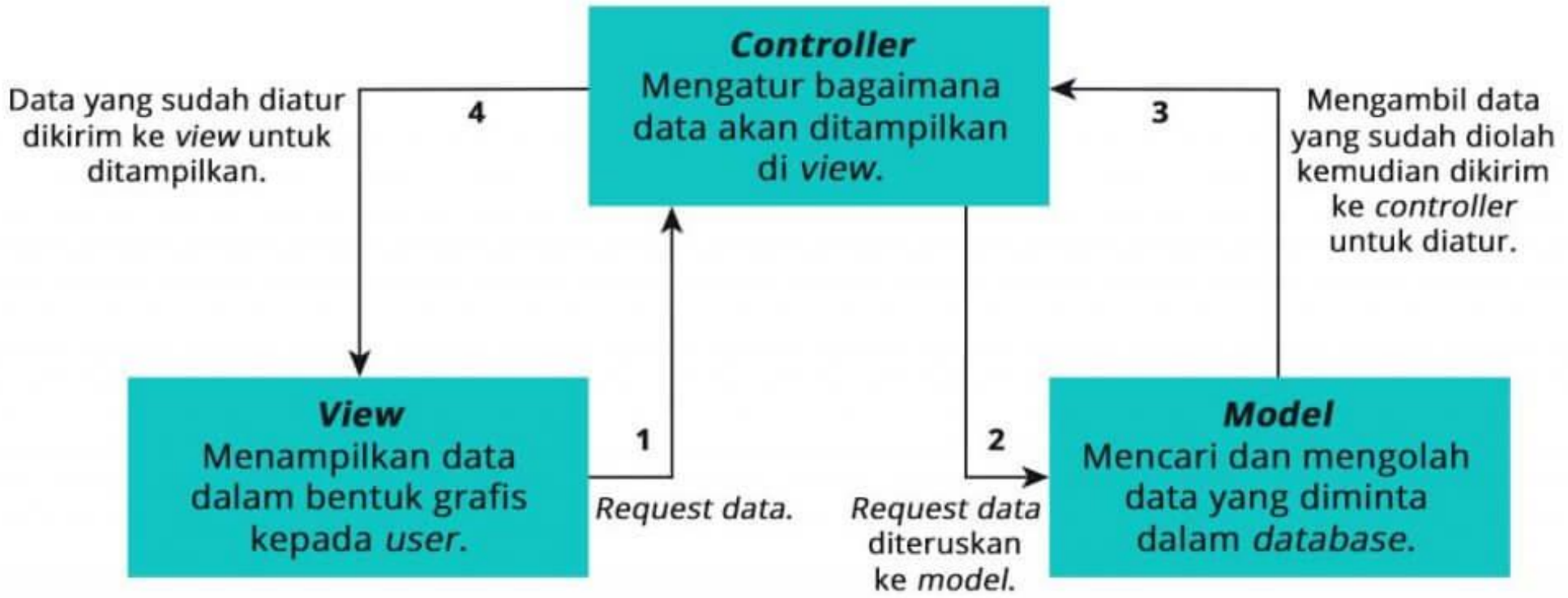
View

- Tugas utama: Menampilkan data kepada pengguna dalam bentuk antarmuka (UI).
- Berinteraksi dengan: Controller (bukan langsung ke Model).
- Contoh:Form login, daftar buku, atau tampilan hasil pencarian di layar pengguna.
- Fungsi utama View:
  - Menyajikan informasi secara visual.
  - Tidak memiliki logika bisnis.
  - Mengupdate tampilan ketika data Model berubah.

Controller

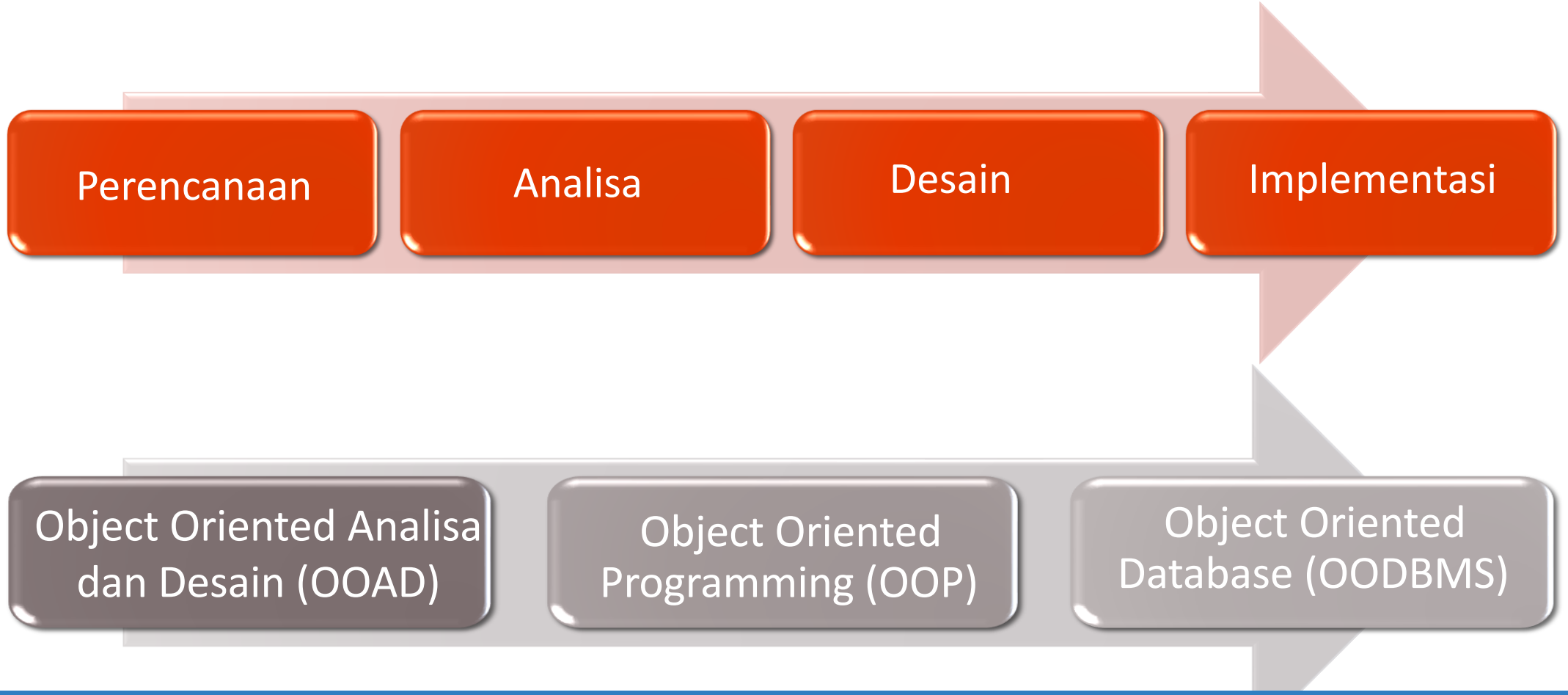
- Tugas utama: Mengatur aliran data antara Model dan View.
- Berinteraksi dengan: Pengguna (melalui input), Model, dan View.
- Contoh:Ketika pengguna menekan tombol “Cari Buku”, Controller menerima input kata kunci, meminta data dari Model, lalu menampilkan hasilnya melalui View.
- Fungsi utama Controller:
  - Menerima dan memproses input dari pengguna.
  - Memanggil fungsi dari Model untuk mengambil atau mengubah data.
  - Memilih View yang tepat untuk ditampilkan kepada pengguna.

Alur



https://www.dicoding.com/

Object Oriented



Target Pembahasan

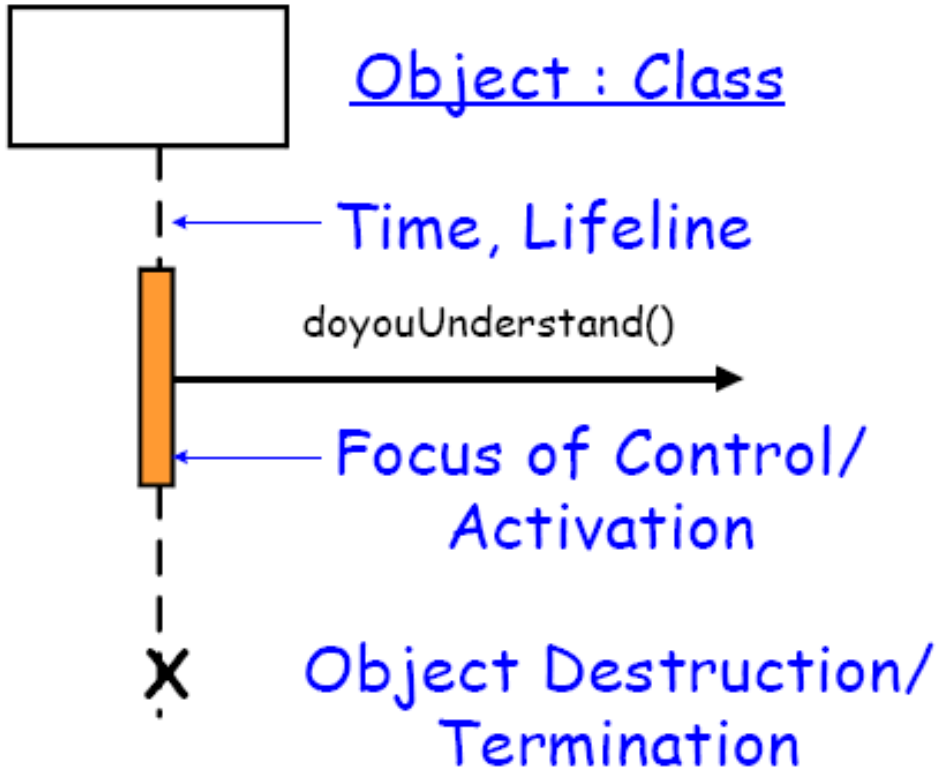
1. Review SDLC
2. Konsep Object – Oriented (Model-View-Controller)
3. Sequence Diagram
4. Class Diagram
5. Praktek

Sequence Diagram (SD)

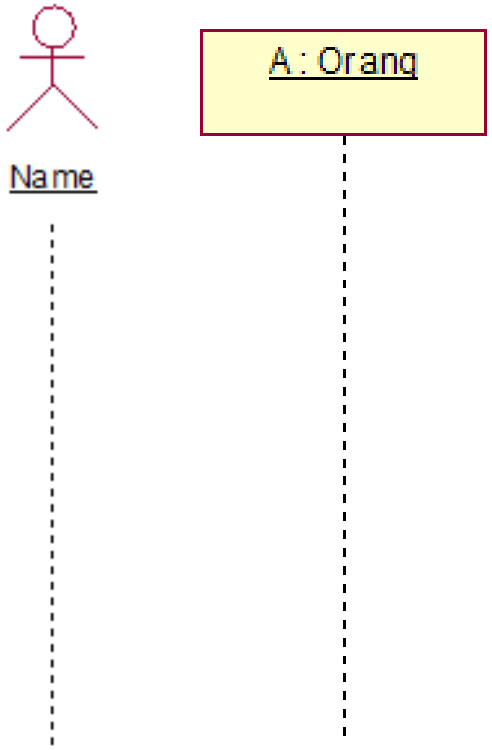
- Menggambarkan perilaku pada sebuah scenario.
- Menunjukkan sejumlah contoh objekyang berpartisipasi dalam sebuah usecase dan message pesan yang melewati obyek-obyek tersebut dalam sebuah waktu untuk sebuah usecase.
- Karena sequence diagram menekankan pada urutan berdasarkan waktu dari sebuah aktivitas di antara seperangkat obyek, maka diagram ini sangat sesuai untuk membantu memahami kebutuhan riil time dan usecase yang kompleks

SD : Terdiri dari...

- Object
- Message
- Time
  - Activation
  - Termination



SD : Actor & Object



Actor

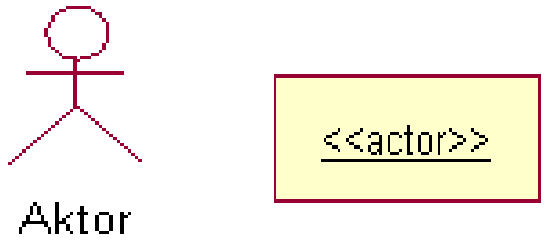
- Eksternal sistem
- Yang berinteraksi dengan sistem

Object

- Diciptakan dari class
- Internal sistem
- nama_object : nama_class
- Contoh A : Orang

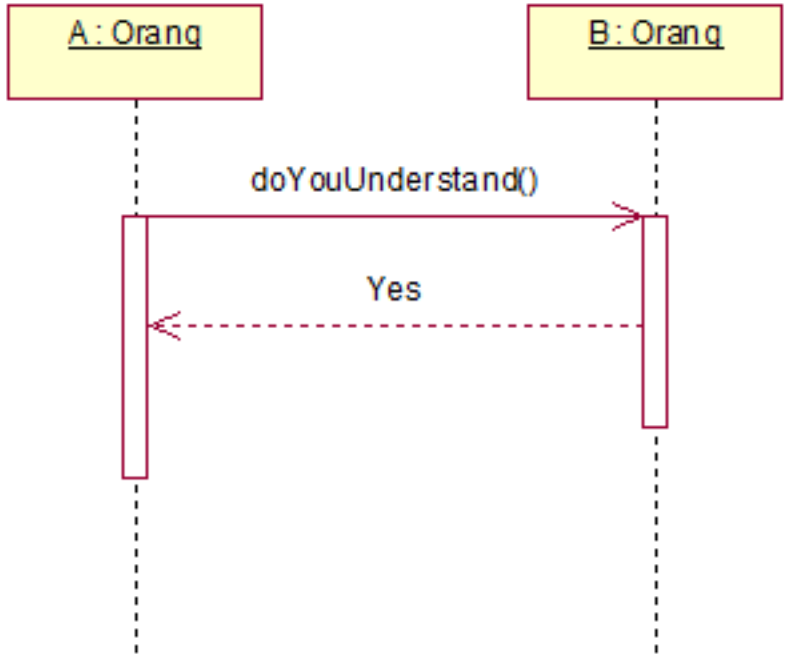
Aktor

- Merupakan orang atau sistem yang mendapatkan keuntungan dari sistem
- Merupakan bagian luar sistem
- Berpartisipasi dalam sequence diagram dengan mengirimkan atau menerima pesan
- Diletakkan pada posisi paling atas dari diagram
- Apabila berupa sistem maka digambarkan dengan bentuk persegi panjang dan keterangan <<aktor>> di dalamnya.

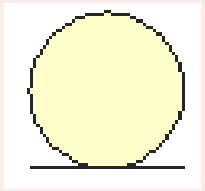
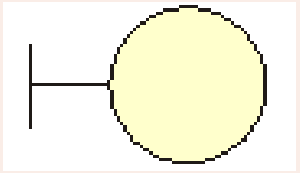
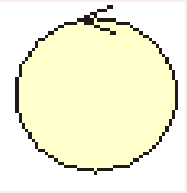


SD : Message & Lifeline

- Message
  - Menyampaikan informasi pada object lain
  - Contoh : doYouUnderstand()
- Return Message
  - Respon dari message object lain
  - Contoh : Yes
- Lifeline
  - Periode waktu aktifnya sebuah object

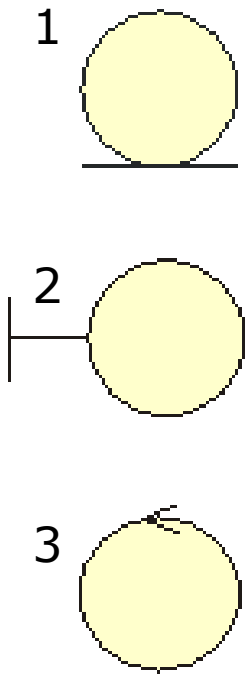


Stereotype Class

PERBEDAAN MODEL VIEW CONTROLLER			
Nama dalam MVC	MODEL	VIEW	CONTROLLER
Nama di Sequence	ENTITY	BOUNDARY	CONTROL
Fungsi	Mengatur, memanggil, dan mengelola data base	memodelkan interaksi di antara actor dan system	Tempat semua Aturan bisnis diletakkan
Isi	Berisi Fungsi : AddBuku(), DeleteBuku(), FindBuku()	Halaman window, Form input, halaman menu	Berisi Fungsi : AddBuku(), DeleteBuku(), FindBuku()
Aturan	Hanya bisa berinteraksi dengan controller	Bisa berinteraksi dengan controller dan Aktor	Bisa berinteraksi Vie dan Model
Notasi Dalam Sequence			

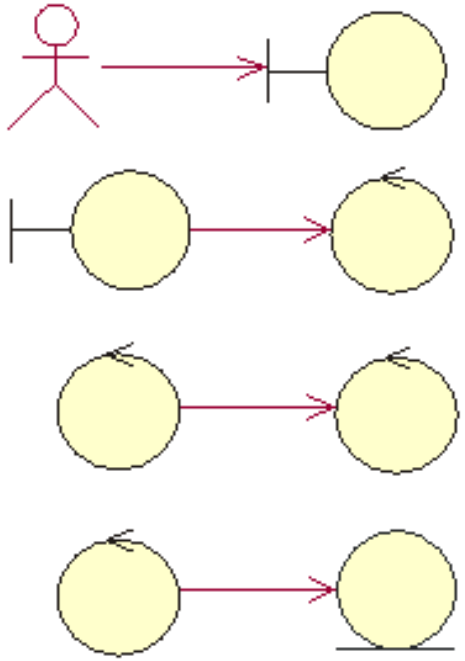
Stereotype class...

- MVC(Model-View-Controler) patteri
  1. Model
     → Menampung data
     → SD : entity
  2. View
     → Menampilkan data
     → SD : boundary
  3. Controller
     → Memanipulasi data
     → SD : control

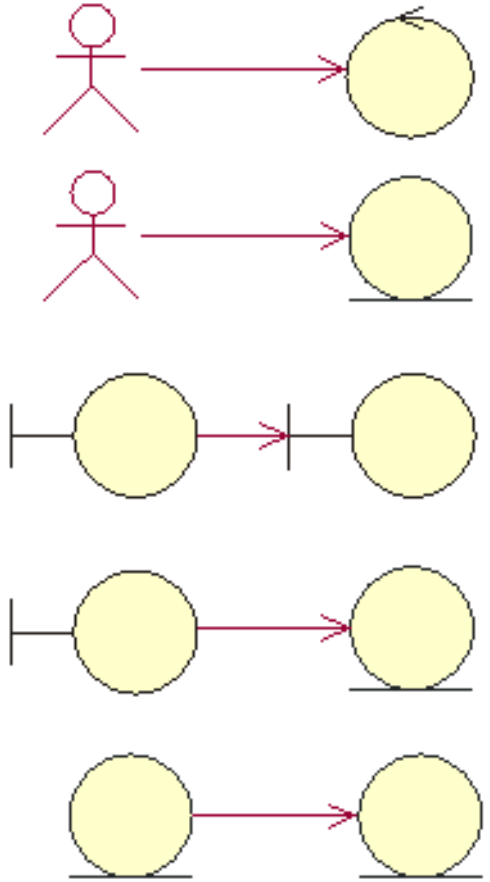


Aturan Sequence Diagram

OK



NOT OK



Target Pembahasan

1. Review SDLC
2. Konsep Object – Oriented (Model-View-Controller)
3. Sequence Diagram
4. Class Diagram
5. Praktek

Class Diagram

Cara memodelkan class dan hubungan antar class dalam system

Static modeling → Attribute → data

- *What the system is*
- Fokus pada “nouns” sistem

Dynamic modeling → behaviour → Method / Function

- *What the system do*
- Fokus pada “verb” sistem

Cara pemodelan sistem paling sesuai untuk developer

Class merepresentasikan object dalam bisnis

Simbol Class Diagram

Class

→ Nama class

→ Atribute/ instance variabel

→ Behavior/ operasi

Visibility

+ Public

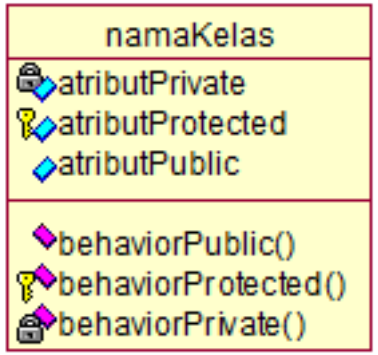
: sifatnya tidak tersembuyi dari umum

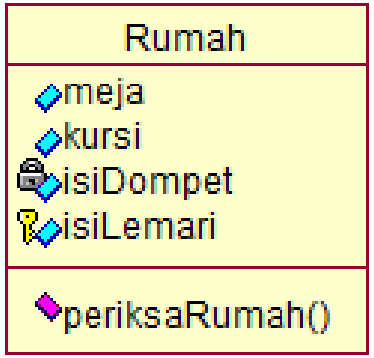
- Private

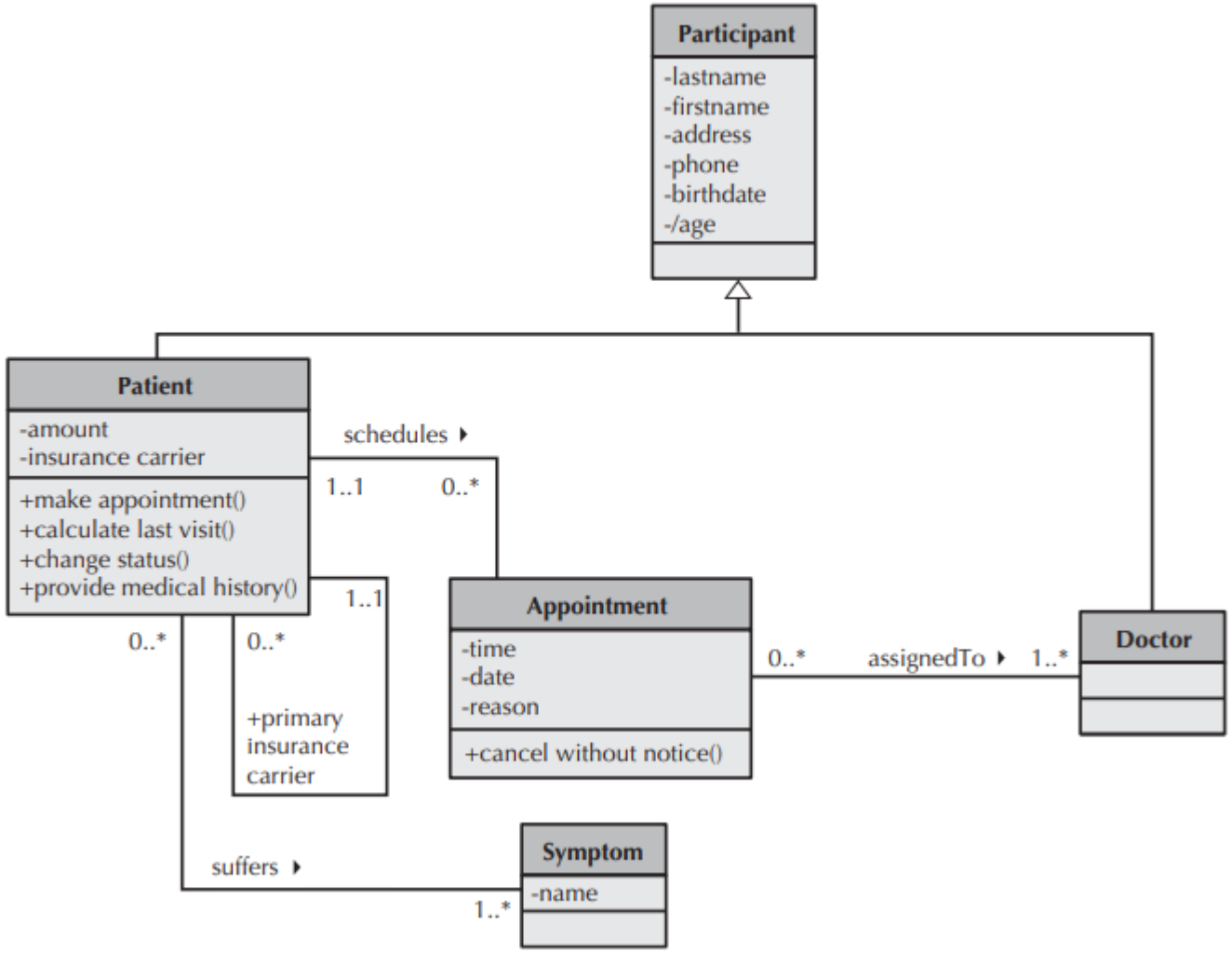
: sifatnya tersembunyi dari umum

Protected

: sifatnya tersembunyi dari umum kecuali subclassnya






Participant
-lastname
-firstname
-address
-phone
-birthdate
-/age
Patient
-amount
-insurance carrier
+make appointment()
+calculate last visit()
+change status()
+provide medical history()
schedules
1..1
0..*
1..1
0..*
0..*
+primary insurance carrier
Appointment
-time
-date
-reason
+cancel without notice()
0..*
assignedTo
1..*
Doctor
suffers
Symptom
-name
1..*

Simbol Class Diagram

Association

→ hubungan antar class

Generalization

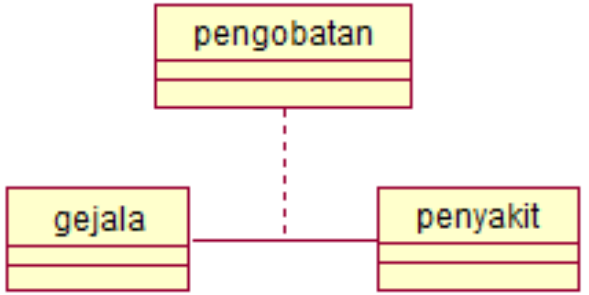
→ turunan

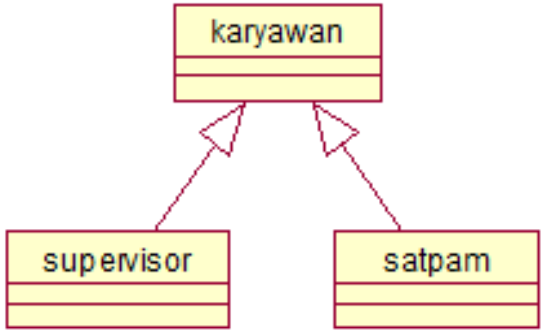
→ is kind of

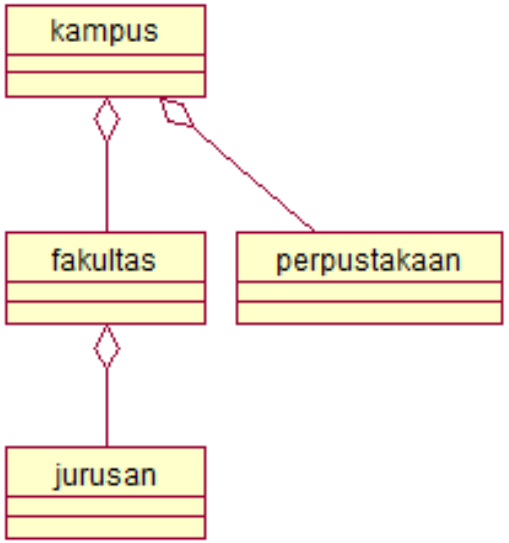
Aggregation

→ bagian

→ is part of





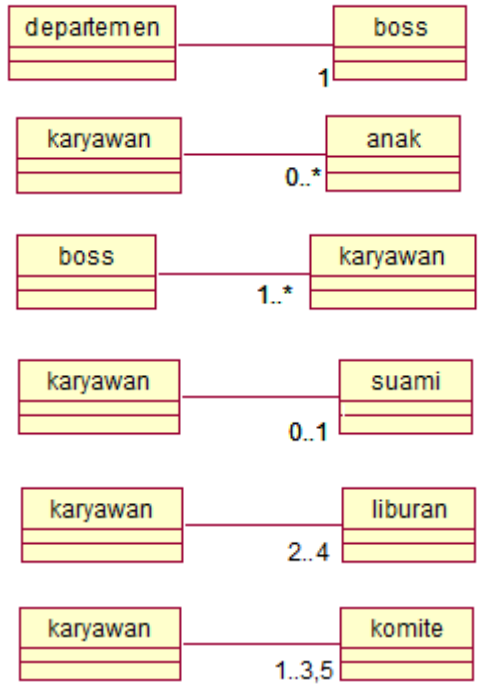


Simbol Class Diagram

Mutiplicity

→ Berapa banyak object yang dapat berpartisi dalam hubungan tersebut.

Exacly one	1	
Zero or more	0..*	
One or more	1..*	
Zero or one	0..1	
Specified range		2..4
Multiple disjoint range		1..3,5



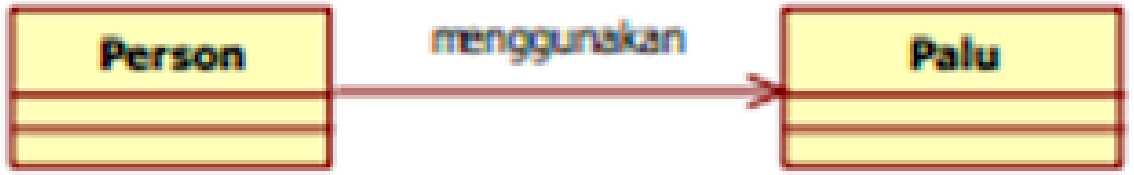
Association

- Dapat diartikan sebagai relasi "..has a.." atau MEMILIKI.
- Digambarkan sebagai garis lurus antara Dua kelas.
- Namun tidak berarti bahwa kelas satu memiliki/dimiliki kelas yang lain, tetapi kelas lain dapat berelasi juga dengan kelas yang sama.



Directed Association

- Relasi seperti asosiasi namun menggambarkan objek atau aliran kejadian berasal dari salah satu kelas, sedang kelas yang lainnya bersifat pasif.



Aggregation

- Dapat diartikan sebagai “..owns a ..”(“terdiri atas..”).
- Terdapat kelas sebagai part clas

  (kelas bagian) yang merupakan bagian dari kelas lain(whole class).
- Namun jika whole class tidak ada, part

  class masih dapat berdiri sendiri.
- Apabila kelas person dihilangkan, maka alamat rumah masih dapat

  berdiri sendiri.

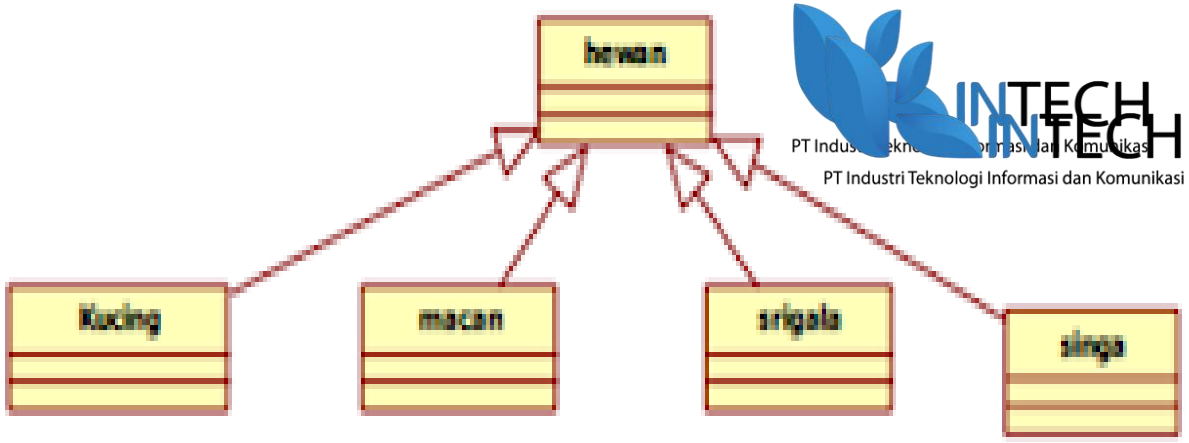


Composition

- Bisa disebut juga sebagai strong agregation, dapat diartikan "..is part of.." ("..bagian dari..").
- Seperti halnya relasi agregasi, namun apabila whole class hilang, maka mustahil part class untuk ada.
- Apabila kelas mobil dihilangkan, maka tangki BBM mobil ikut hilang.



Generalization



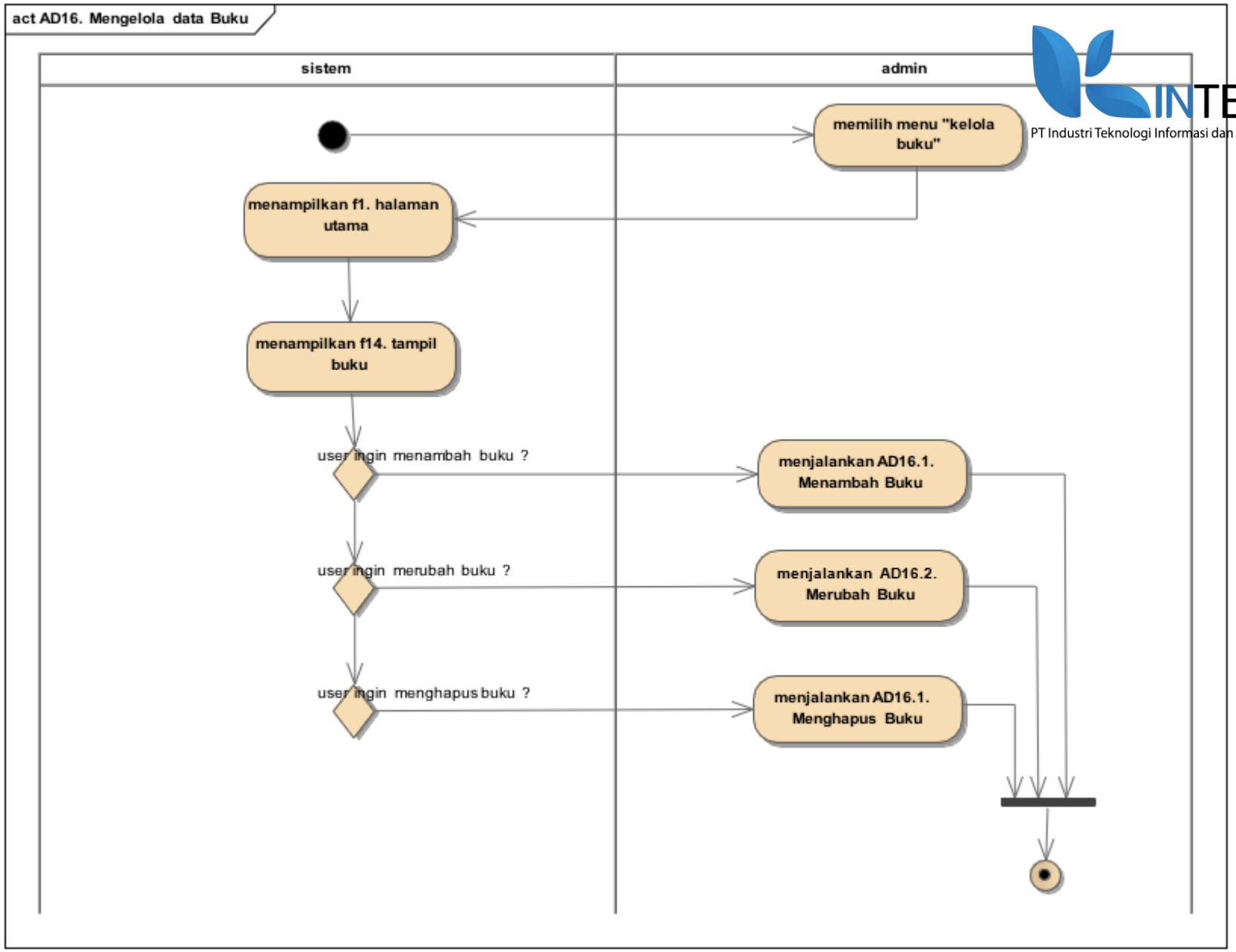
- Dapat diartikan sebagai relasi “..is a..” Digunakan untuk merepresentasikan pewarisan.
- Suatu kelas (childclass) dapat diturunkan dari kelas lain dan mewarisi semua atribut dan method induknya (parentclass) dan dapat menambah method atau atribut baru.
- Sebagai contoh kucing, srigala, singa, macan adalah hewan.
- Maka bisa dibuat relasi antar kelas seperti di atas:

Target Pembahasan

1. Review SDLC
2. Konsep Object – Oriented (Model-View-Controller)
3. Sequence Diagram
4. Class Diagram
5. Praktek

Latihan 1

Dari Hasil Analisa Sistem Informasi perpustakaan, Buat Sequence Diagram untuk Activity Diagram : Mengelola Buku

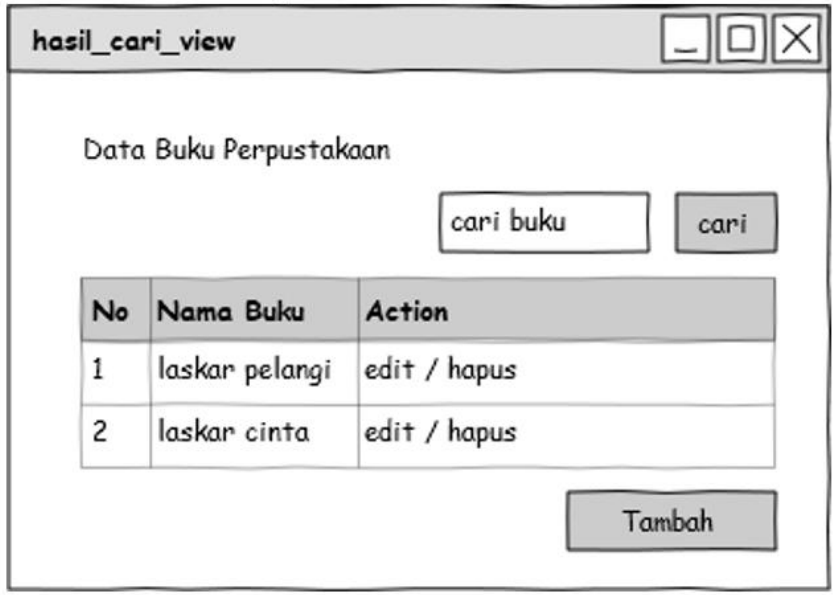


INTECH
PT Industri Teknologi Informasi dan Komunikasi

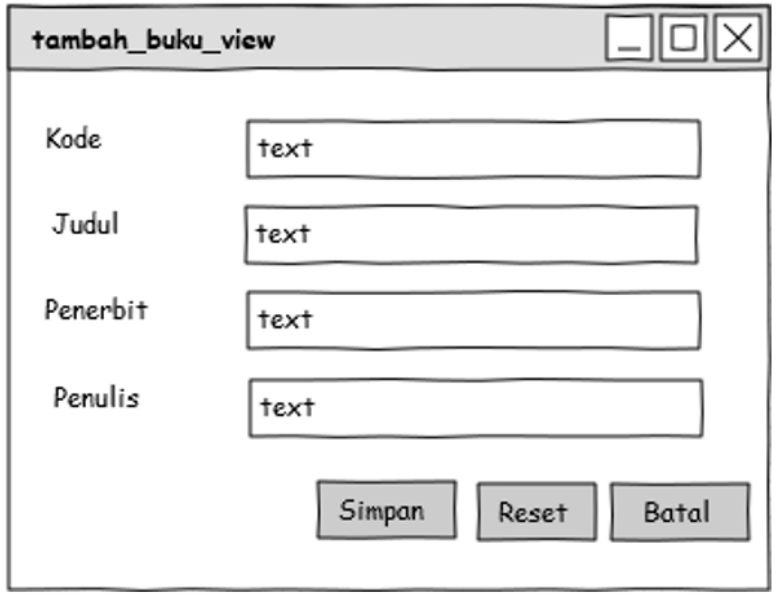
F1. Halaman utama



F14. Tampil Buku



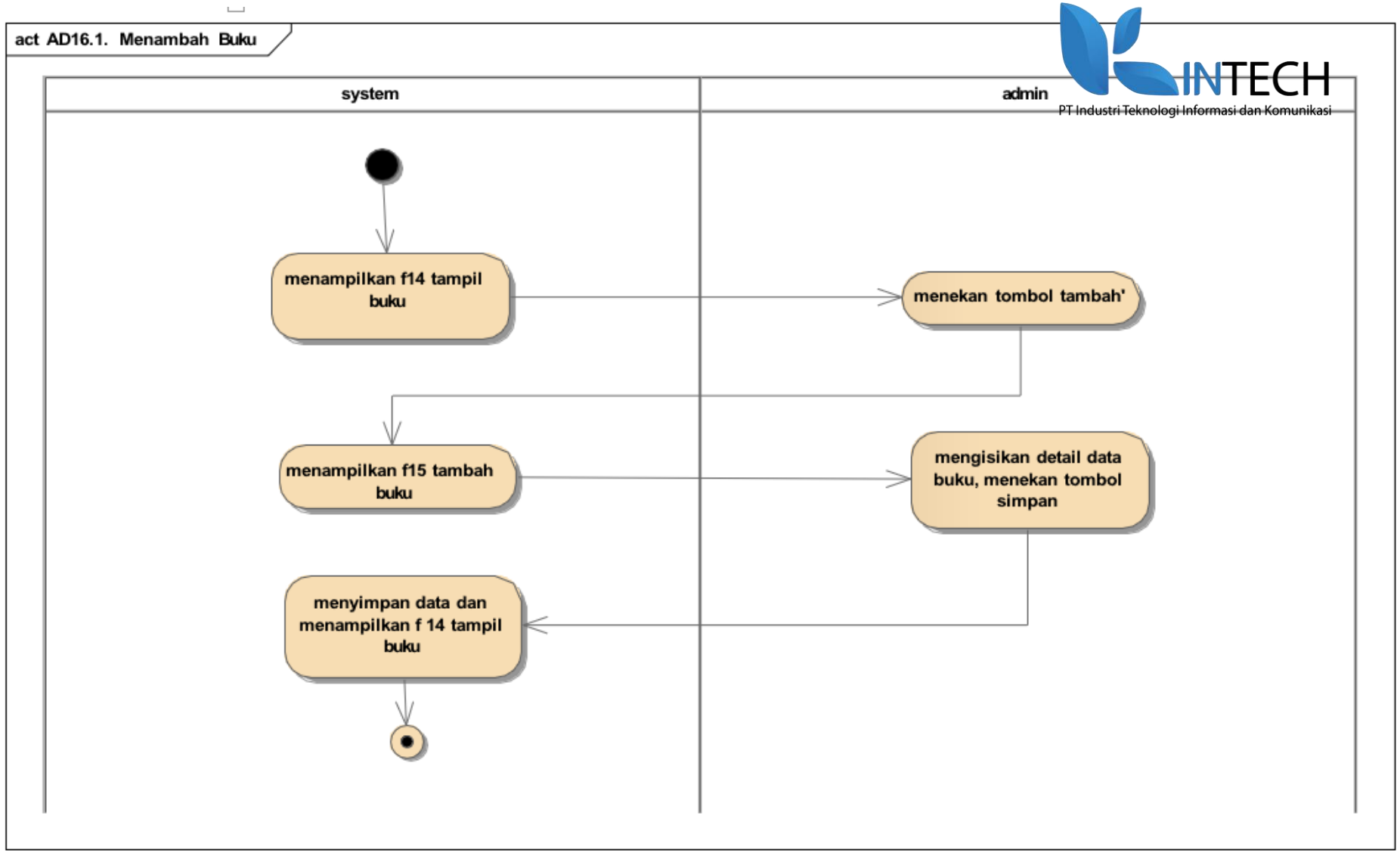
F15 tambah buku



Latihan 2

Dari Hasil Analisa Sistem Informasi perpustakaan, Buat Sequence Diagram untuk Activity Diagram :

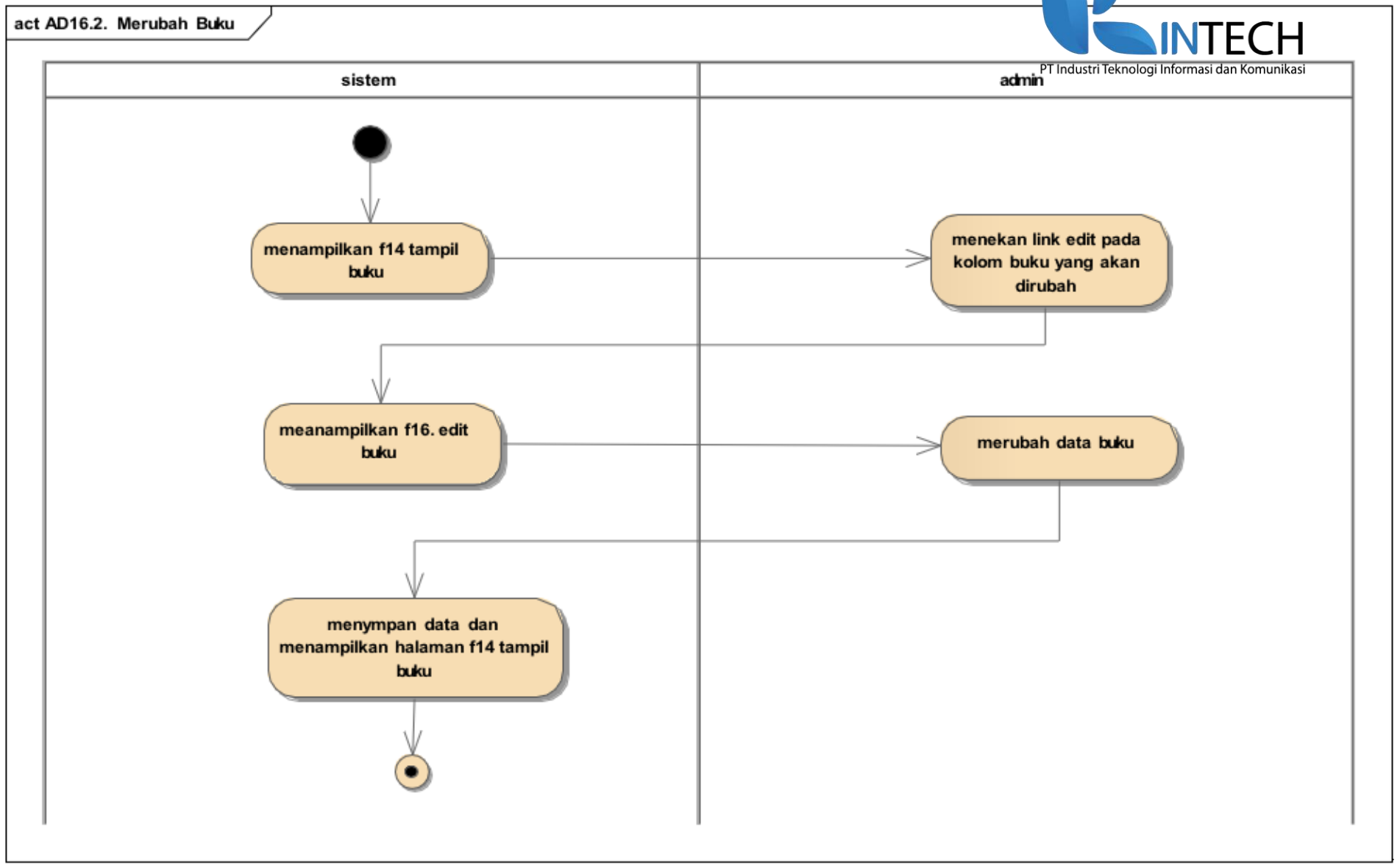
Menambah Buku



Latihan 3

Dari Hasil Analisa Sistem Informasi perpustakaan, Buat Sequence Diagram untuk Activity Diagram :

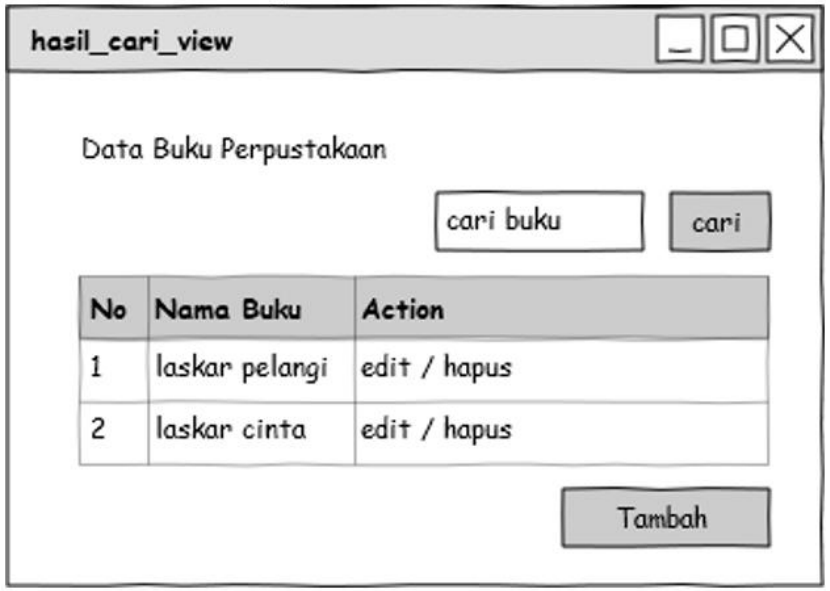
Merubah Buku



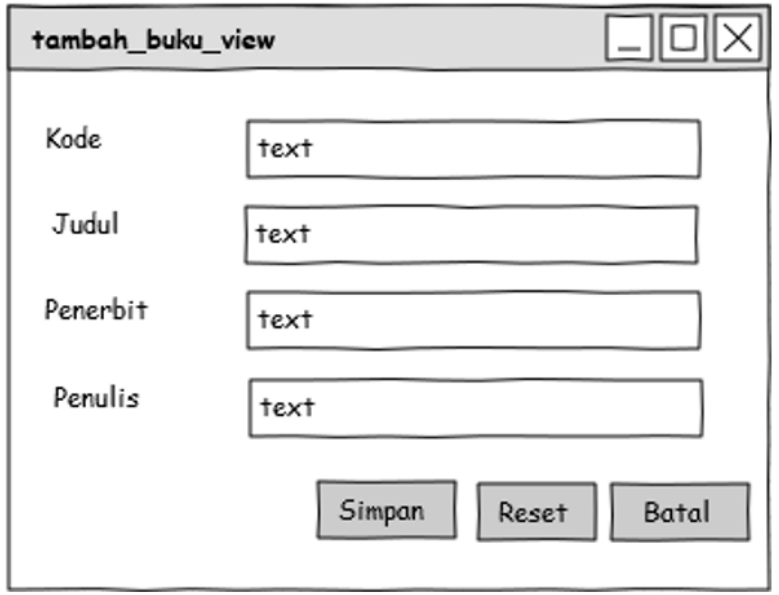
F1. Halaman utama



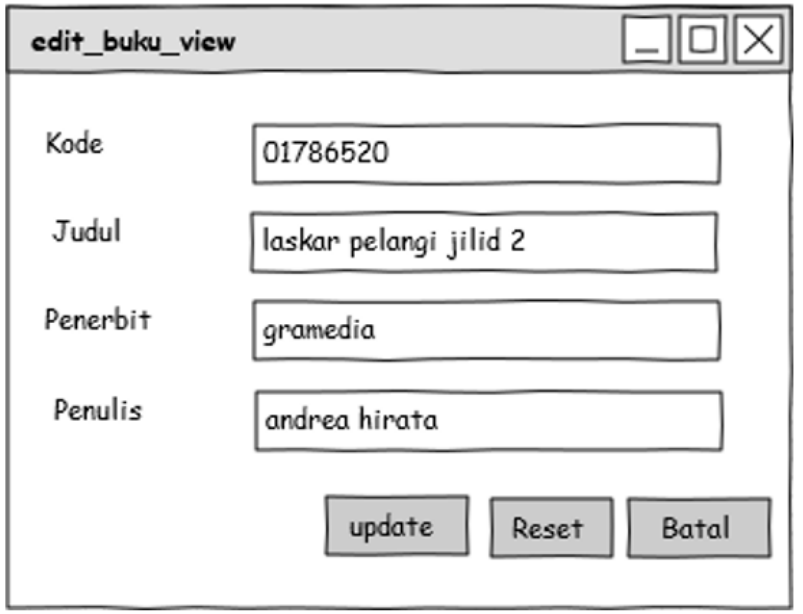
F14. Tampil Buku



F15 tambah buku



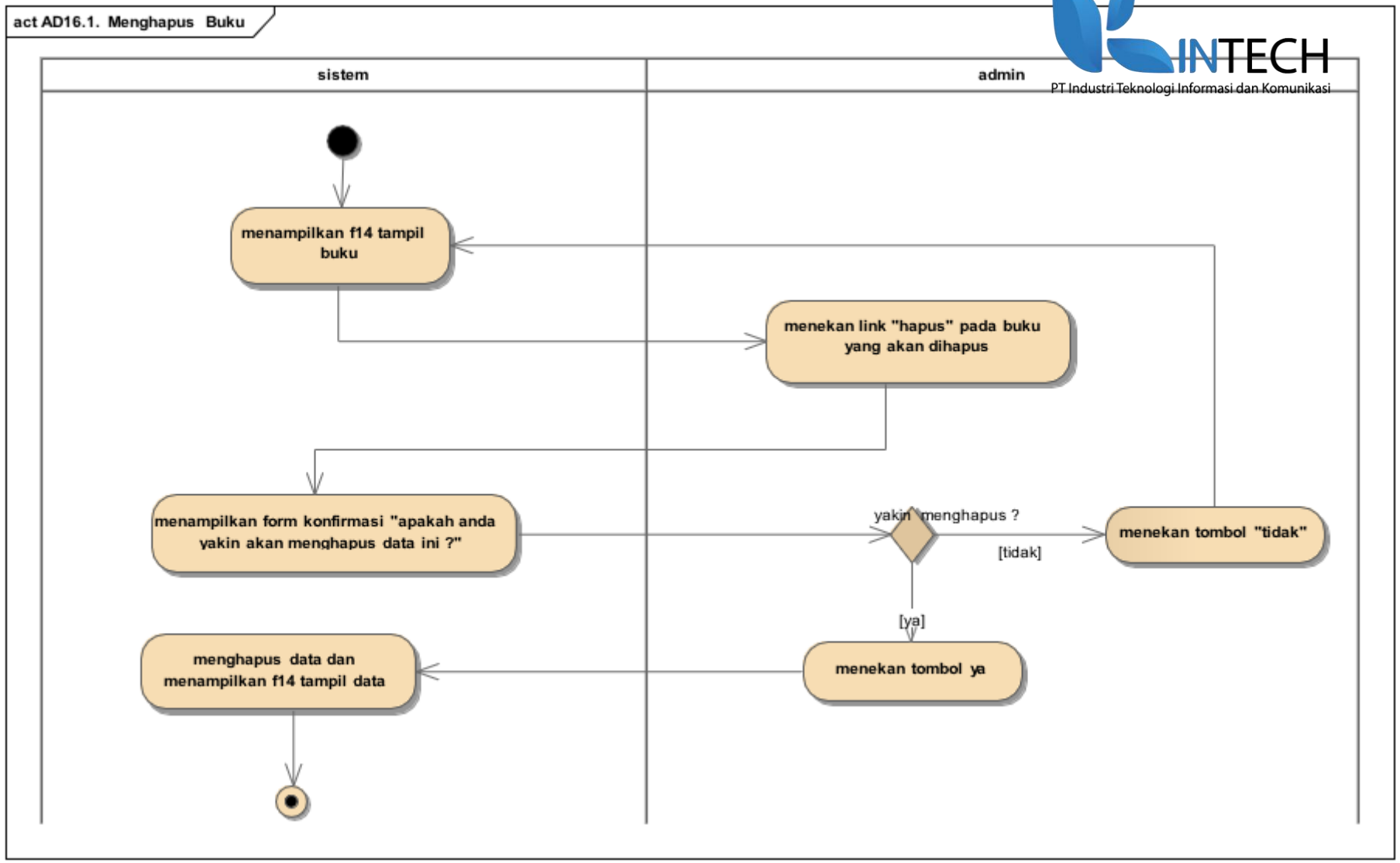
F16. Edit Buku



Latihan 4

Dari Hasil Analisa Sistem Informasi perpustakaan, Buat Sequence Diagram untuk Activity Diagram :

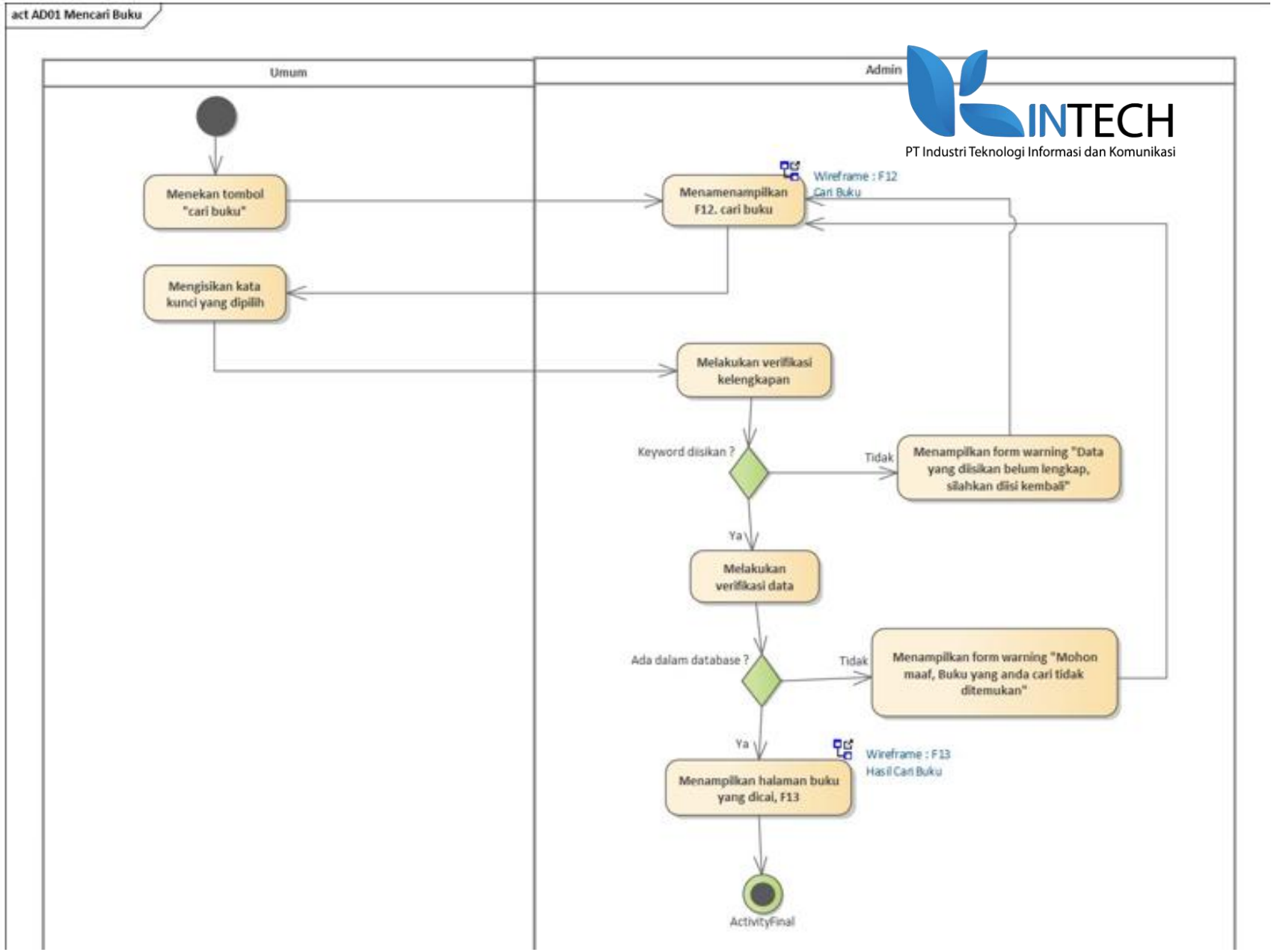
Menghapus Buku



Latihan 5

Dari Hasil Analisa Sistem Informasi perpustakaan, Buat Sequence Diagram untuk Activity Diagram :

Mencari Buku

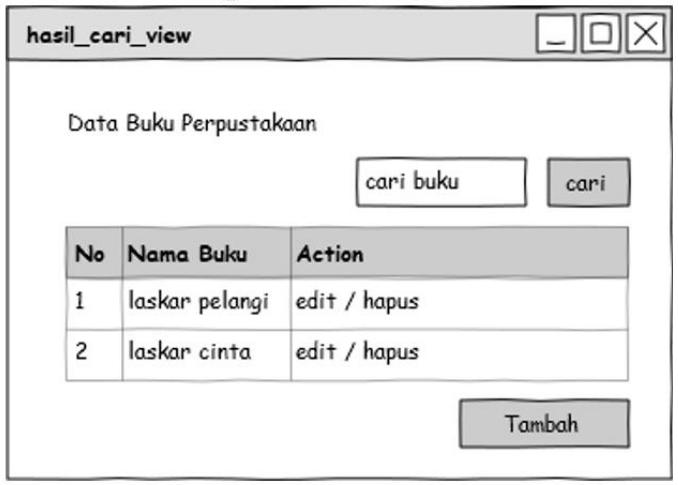


F1. Halaman utama

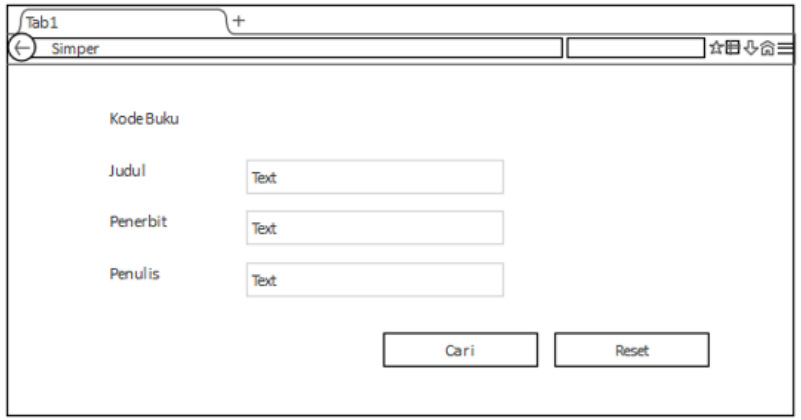


INTECH
PT Industri Teknologi Informasi dan Komunikasi

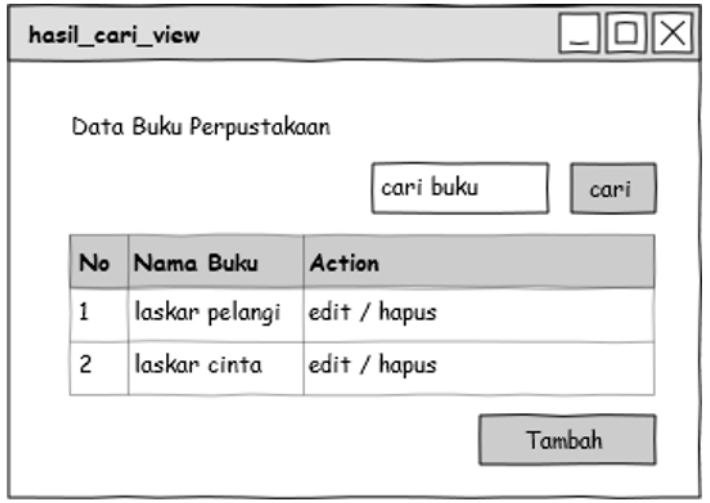
F14. Tampil Buku



F.12 Halaman Cari Buku



F13. Halaman Buku yang dicari




thank you
danke
謝謝
ngiyabonga
teşekkür ederim
gracias
спасибо
dziękuję
bedankt
obrigado
merci
go raibh maith agat
mochchakkeram
tapadh leat
dank je
sukriya
kop khun krap
grazie
arigatô
dakujem
terima kasih
감사합니다
ευχαριστώ
мерси
баярлалаа
рахмат
hvala
mauruuru
sagolun
takk
taiku
dankie
kaitos
matondo
misaotra
blagodaram
paldies
asante
manana
mahalo
barka
faafetai
lava
vinaka
dhanyavadagalu
xiexie
shukriya
trugarez
merci